

Intégration Firebase Storage

Où vont les images et documents téléversés dans l'admin, comment le code décide du stockage, et comment configurer, exploiter et dépanner Firebase Storage sur l'hébergement Render.

FOURNISSEUR Firestore Storage = bucket Google Cloud Storage	OFFRE Spark (gratuite) — ~5 Go stockés, ~1 Go / jour téléchargé
BACKEND DJANGO storages.backends.gcloud.GoogleCloudStorage	EMPLACEMENT DES FICHIERS bucket, sous le préfixe media/
TYPE D'URL SERVIE URL signée V4, valable 24 h, régénérée à chaque rendu	DÉVELOPPEMENT LOCAL inchangé — disque koyra/media/, servi par Django

§1 Pourquoi Firebase

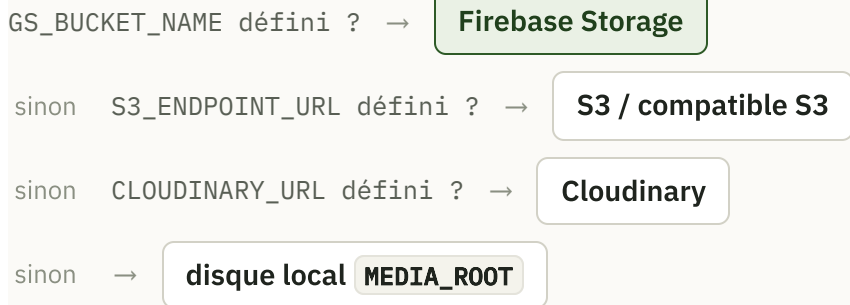
Sur l'offre gratuite de Render, le disque du serveur est **éphémère** : tout fichier téléversé dans l'admin disparaît au prochain redéploiement ou à la mise en veille. Il faut donc un stockage **externe**.

Cloudinary, envisagé d'abord, **bloque les inscriptions depuis le Mali**. Firebase Storage n'a pas cette restriction : c'est un service Google, disponible partout. Sous le capot, un « bucket » Firebase *est* un bucket Google Cloud Storage — un simple entrepôt de fichiers, qui accepte aussi bien les images que les PDF ou documents Word.

CRITÈRE	FIREBASE STORAGE
Gratuit	~5 Go de stockage, ~1 Go / jour de téléchargement (offre Spark)
Carte bancaire	non requise pour Spark
Types de fichiers	quelconques — images <i>et</i> documents
Disponible au Mali	oui (Google)
Intégration Django	backend standard <code>django-storages</code>

§2 La chaîne de stockage

Dans `koyra_web/settings.py`, un bloc unique choisit le stockage par défaut de Django selon les **variables d'environnement présentes**. Le premier fournisseur configuré gagne ; sinon on retombe sur le disque local.



BACKEND	DÉCLENCHEUR	RÔLE
Firebase / GCS	<code>GS_BUCKET_NAME</code>	choix actuel en production
S3 (B2, R2, Wasabi...)	<code>S3_ENDPOINT_URL</code>	alternative prête, non utilisée
Cloudinary	<code>CLOUDINARY_URL</code>	indisponible depuis certains pays
Disque local	aucune variable	développement, ou Render payant + disque

En parallèle, `koyra_web/urls.py` ne branche la route qui sert `/media/...` depuis le disque **que si aucun** stockage externe n'est configuré. Avec Firebase, les fichiers ont une URL Google : rien ne transite par le serveur Django.

§3 Ce qui se passe à l'exécution

Au téléversement (admin)

- 1 Dans l'admin, on ajoute une image sur un produit et on enregistre.
- 2 Le champ `ImageField` appelle `save()` sur le stockage par défaut — ici `GoogleCloudStorage`.
- 3 Le backend authentifie l'appel avec la **clé de compte de service** et envoie les octets vers `gs://<bucket>/media/produits/<nom-fichier>`.
- 4 Le chemin relatif (`produits/photo.jpg`) est enregistré en base ; le préfixe `media/` est ajouté par `GS_LOCATION`.

À l'affichage (site public)

- 1 Le gabarit rend `{{ produit.image.url }}`.
- 2 Le backend fabrique une **URL signée V4** :
`https://storage.googleapis.com/<bucket>/media/produits/photo.jpg?X-Goog-Algorithm=...&X-Goog-Signature=...`, valable 24 h.
- 3 Le navigateur du visiteur charge l'image **directement chez Google**, sans passer par Render.
- 4 Au prochain rendu de la page, une **nouvelle** URL signée de 24 h est produite. Le site sert donc toujours un lien valide, même si le précédent a expiré.

§4 Les URLs signées

Rendre un bucket Firebase lisible publiquement demande de désactiver plusieurs protections Google (accès uniforme, *public access prevention*) — fragile, et facile à rater. On sert donc des **URLs signées** : le backend signe chaque lien avec la clé privée du compte de service.

- **Aucun réglage** côté Firebase / Google Cloud : ni règles Storage, ni IAM. Seule la clé compte.
- Le bucket **reste privé** : sans lien signé émis par le site, un fichier n'est pas accessible.
- Expiration **24 h** (`GS_EXPIRATION`). Comme les pages sont rendues côté serveur à chaque visite, le lien est toujours frais au moment où le navigateur le suit.

CONSÉQUENCE

Ne pas **copier ni mettre en cache** ces URLs ailleurs (e-mail, flux RSS, export figé) : elles cessent de fonctionner après 24 h. Pour un lien durable vers un fichier, il faudra passer par un accès public au bucket ou un domaine personnalisé.

§5 Les réglages dans settings.py

Branche activée dès que `GS_BUCKET_NAME` est présent :

```
# 1) Firebase Storage (= bucket Google Cloud Storage)
if GS_BUCKET_NAME:
    _raw = os.getenv('GS_CREDENTIALS_JSON', '')
    try:
        _info = json.loads(_raw)          # JSON brut
    except json.JSONDecodeError:
        _info = json.loads(base64.b64decode(_raw)) # ou base64

    INSTALLED_APPS += ['storages']
    STORAGES['default'] = {'BACKEND': 'storages.backends.gcloud.GoogleCloudStorage'}
    GS_CREDENTIALS = service_account.Credentials.from_service_account_info(_info)
    GS_PROJECT_ID = _info.get('project_id')
    GS_DEFAULT_ACL = None
    GS_QUERYSTRING_AUTH = True           # -> URLs signées
    GS_EXPIRATION = timedelta(hours=24)
    GS_FILE_OVERWRITE = False
    GS_LOCATION = 'media'
```

RÉGLAGE	VALEUR	EFFET
<code>STORAGES['default']</code> <code>]</code>	<code>GoogleCloudStorage</code>	tout <code>FileField</code> / <code>ImageField</code> passe par le bucket
<code>GS_CREDENTIALS</code>	objet clé de service	authentifie les envois et signe les URLs
<code>GS_LOCATION</code>	<code>'media'</code>	préfixe de tous les fichiers dans le bucket
<code>GS_QUERYSTRING_AUTH</code> <code>H</code>	<code>True</code>	génère des liens signés plutôt que publics
<code>GS_EXPIRATION</code>	<code>timedelta(hours=24)</code> <code>)</code>	durée de validité d'un lien signé
<code>GS_FILE_OVERWRITE</code>	<code>False</code>	deux fichiers de même nom → suffixe auto, pas d'écrasement
<code>GS_DEFAULT_ACL</code>	<code>None</code>	aucune ACL posée (accès géré par la signature)

§6 Les deux variables d'environnement

VARIABLE	CONTENU	OÙ LA TROUVER
GS_BUCKET_NAME	nom du bucket, ex. <code>koyra-1a2b3.firebaseio.com</code> (ou <code>...appspot.com</code>)	Console Firebase → Storage , affiché en haut de la page
GS_CREDENTIALS_JSON	tout le contenu du fichier <code>.json</code> de compte de service — JSON brut collé tel quel, <i>ou</i> ce même JSON encodé en base64 (le code teste les deux)	Console Firebase → ⚙ Paramètres du projet → Comptes de service → <i>Générer une nouvelle clé privée</i>

En production : onglet **Environment** du service `koyra` sur Render. **En local** : fichier `.env` à la racine (voir `.env.example`) — laissées vides, le stockage reste local.

ASTUCE BASE64

Si le collage multi-lignes du JSON pose problème sur Render, encoder d'abord :

```
base64 -w0 cle.json
```

 (Linux/macOS) ou

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("cle.json"))
```

(PowerShell), puis coller la chaîne obtenue.

§7 Mise en place

- 1 **Créer le projet + le bucket.** console.firebase.google.com → *Ajouter un projet* → menu **Créer** → **Storage** → **Commencer** → emplacement (ex. `europa-west`) → mode *production*. Noter le nom du bucket.
- 2 **Ne rien changer aux règles Storage.** Le bucket reste privé ; les URLs signées suffisent.
- 3 **Générer la clé.** ⚙ *Paramètres du projet* → *Comptes de service* → *Générer une nouvelle clé privée* → un fichier `.json` se télécharge.
- 4 **Renseigner Render.** Service `koyra` → *Environment* → ajouter `GS_BUCKET_NAME` et `GS_CREDENTIALS_JSON` → *Save* (redéploiement ~4–5 min, installation des bibliothèques Google).
- 5 **Re-téléverser les images existantes.** Les fiches créées avant Firebase pointaient vers le disque éphémère : rouvrir chaque produit / marque / catégorie et remettre l'image.

VÉRIFIER

Sur une fiche en ligne, inspecter la balise `<img>` : l'attribut `src` doit être `https://storage.googleapis.com/<bucket>/media/...?X-Goog-Signature=...`. Si oui, l'intégration fonctionne.

§8 Développement local

Sans `GS_BUCKET_NAME` dans l'environnement, la chaîne (§2) retombe sur le disque : les fichiers vont dans `koyra/media/` et sont servis par Django via la route `/media/`.
`python manage.py runserver` se comporte exactement comme avant l'intégration.

Pour tester Firebase *en local*, il suffit de mettre les deux variables dans le `.env` — le même code s'active alors.

§9 Fichiers modifiés

FICHIER	CHANGEMENT
<code>koyra_web/setting</code> <code>s.py</code>	chaîne de stockage + branche Firebase (URLs signées 24 h, préfixe <code>media/</code>)
<code>koyra_web/urls.p</code> <code>y</code>	route <code>/media/</code> locale seulement si aucun stockage externe
<code>requirements.txt</code>	+ <code>django-storages</code> , <code>google-cloud-storage</code> , <code>google-auth</code> (et leurs dépendances : <code>cryptography</code> , <code>protobuf</code> ...)
<code>render.yaml</code>	<code>GS_BUCKET_NAME</code> + <code>GS_CREDENTIALS_JSON</code> déclarées (<code>sync: false</code>)
<code>.env.example</code>	modèle des variables, avec les alternatives S3
<code>DEPLOIEMENT.md</code>	§6 : guide pas-à-pas Firebase

Commits : `a6f1e71` (backend Firebase + URLs signées) et `23b5f56` (socle `django-storages` compatible S3).

§10 Quotas & exploitation

Offre **Spark** (gratuite), ordres de grandeur : **5 Go** stockés, **1 Go / jour** de téléchargement, plusieurs dizaines de milliers d'opérations par jour. Pour un catalogue, cela représente des milliers de photos et documents.

- **Suivi** : Console Firebase → *Storage* → onglet *Usage*.
- **Alerte** : si le quota de téléchargement journalier est atteint, les images renvoient une erreur jusqu'au lendemain.
- **Si ça déborde** : passer le projet en *Blaze* (paiement à l'usage, quelques centimes par Go au-delà du gratuit) — aucun changement de code.

§11 Dépannage

SYMPTÔME	CAUSE PROBABLE	CORRECTIF
Images OK en local, cassées en ligne	<code>config</code> variables absentes sur Render	vérifier <code>GS_BUCKET_NAME</code> et <code>GS_CREDENTIALS_JSON</code> dans <i>Environment</i> , puis <i>Manual Deploy</i>
Au démarrage : <code>ValueError</code> / <code>Could not deserialize key</code>	<code>clé</code> JSON tronqué ou mal collé	recoller tout le fichier <code>.json</code> , ou utiliser la version base64
Envoi d'image → <code>403</code> / <code>Forbidden</code>	<code>droits</code> compte de service sans accès Storage	utiliser la clé générée par Firebase (elle a le rôle Storage Admin), pas une clé GCP quelconque
Envoi → <code>404</code> / bucket introuvable	<code>nom</code> <code>GS_BUCKET_NAME</code> erroné	copier le nom exact affiché dans Storage (<code>...firebasestorage.app</code> ou <code>...appspot.com</code>)
<code></code> pointe vers <code>/media/...</code> et non Google	<code>chaîne</code> branche Firebase non prise	<code>GS_BUCKET_NAME</code> vide au démarrage — vérifier l'orthographe de la variable
Images qui marchaient → soudain <code>403</code>	<code>quota</code> plafond journalier de téléchargement atteint	attendre le reset, ou passer en Blaze
Build Render échoue après l'ajout	<code>deps</code> bibliothèques Google manquantes	confirmer que <code>google-cloud-storage</code> est dans <code>requirements.txt</code>

§12 Migrer plus tard

Vers un disque persistant Render (offre payante)

Décommenter le bloc `disk:` de `render.yaml`, définir `MEDIA_ROOT=/var/media`, **retirer** les variables `GS_*` — la chaîne (§2) repasse alors sur le disque. Les fichiers Firebase existants sont à re-téléverser (ou à copier via `gsutil`).

Vers un autre fournisseur compatible S3

Le socle est déjà là : renseigner `S3_ENDPOINT_URL`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `S3_BUCKET_NAME`, `S3_REGION` (Backblaze B2, Cloudflare R2, Supabase...). Retirer `GS_BUCKET_NAME` pour que S3 prenne la main.

Récupérer les fichiers depuis Firebase

Console Firebase → Storage (téléchargement fichier par fichier), ou en masse : `gsutil -m cp -r gs://<bucket>/media ./media`.

Référence interne — dépôt `github.com/aichadev2024/koyra`. Reflète l'état du code au commit `a6f1e71`.